# ASSIGNMENT 8

**Name:** Rohit Kailas Deore
**Roll No:** 11
**Batch:** T1
**Class:** TE-IT

[1]:
```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix, ConfusionMatrixDisplay
)

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

## 1. Load the Wine Dataset

[2]:
```
# Use the built-in dataset; no Kaggle download is required
data = load_wine()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="wine_class")

print("First five rows:")
display(X.head())
print("Dataset shape:", X.shape)
print("Classes:", list(data.target_names))
```

```
First five rows:
```

| | alcohol | malic_acid | ash | alcalinity_of_ash | magnesium | total_phenols | \ |
|---|---|---|---|---|---|---|---|
| 0 | 14.23 | 1.71 | 2.43 | 15.6 | 127.0 | 2.80 | |
| 1 | 13.20 | 1.78 | 2.14 | 11.2 | 100.0 | 2.65 | |
| 2 | 13.16 | 2.36 | 2.67 | 18.6 | 101.0 | 2.80 | |
| 3 | 14.37 | 1.95 | 2.50 | 16.8 | 113.0 | 3.85 | |
| 4 | 13.24 | 2.59 | 2.87 | 21.0 | 118.0 | 2.80 | |

| | flavanoids | nonflavanoid_phenols | proanthocyanins | color_intensity | hue | \ |
|---|---|---|---|---|---|---|
| 0 | 3.06 | 0.28 | 2.29 | 5.64 | 1.04 | |
| 1 | 2.76 | 0.26 | 1.28 | 4.38 | 1.05 | |
| 2 | 3.24 | 0.30 | 2.81 | 5.68 | 1.03 | |
| 3 | 3.49 | 0.24 | 2.18 | 7.80 | 0.86 | |

```
4        2.69                  0.39             1.82             4.32  1.04

   od280/od315_of_diluted_wines  proline
0                          3.92   1065.0
1                          3.40   1050.0
2                          3.17   1185.0
3                          3.45   1480.0
4                          2.93    735.0

Dataset shape: (178, 13)
Classes: [np.str_('class_0'), np.str_('class_1'), np.str_('class_2')]
```

## 2. Split the Dataset

[3]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (142, 13)
Testing data: (36, 13)
```

## 3. Decision Tree Classifier

[4]:
```
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)

print("=== Decision Tree Classification Report ===")
print(classification_report(
    y_test, y_pred_dt, target_names=data.target_names, zero_division=0
))
```

```
=== Decision Tree Classification Report ===
              precision    recall  f1-score   support

     class_0       1.00      0.92      0.96        12
     class_1       0.88      1.00      0.93        14
     class_2       1.00      0.90      0.95        10

    accuracy                           0.94        36
   macro avg       0.96      0.94      0.95        36
weighted avg       0.95      0.94      0.94        36
```

## 4. Gaussian Naïve Bayes Classifier

[5]:
```
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
y_pred_nb = nb_model.predict(X_test)

print("=== Gaussian Naïve Bayes Classification Report ===")
print(classification_report(
    y_test, y_pred_nb, target_names=data.target_names, zero_division=0
))
```

```
=== Gaussian Naïve Bayes Classification Report ===
              precision    recall  f1-score   support

     class_0       0.92      1.00      0.96        12
     class_1       1.00      0.93      0.96        14
     class_2       1.00      1.00      1.00        10

    accuracy                           0.97        36
   macro avg       0.97      0.98      0.97        36
weighted avg       0.97      0.97      0.97        36
```

## 5. Compare Both Classifiers

[6]:
```
results = pd.DataFrame({
    "Model": ["Decision Tree", "Gaussian Naïve Bayes"],
    "Accuracy": [
        accuracy_score(y_test, y_pred_dt),
        accuracy_score(y_test, y_pred_nb)
    ],
    "Precision (weighted)": [
        precision_score(y_test, y_pred_dt, average="weighted", zero_division=0),
        precision_score(y_test, y_pred_nb, average="weighted", zero_division=0)
    ],
    "Recall (weighted)": [
        recall_score(y_test, y_pred_dt, average="weighted", zero_division=0),
        recall_score(y_test, y_pred_nb, average="weighted", zero_division=0)
    ],
    "F1 Score (weighted)": [
        f1_score(y_test, y_pred_dt, average="weighted", zero_division=0),
        f1_score(y_test, y_pred_nb, average="weighted", zero_division=0)
    ]
})

display(results.round(4))
```

```
best_model = results.loc[results["Accuracy"].idxmax(), "Model"]
print("Better model based on accuracy:", best_model)
```

```
                  Model  Accuracy  Precision (weighted)  Recall (weighted)  \
0         Decision Tree    0.9444                0.9514             0.9444
1  Gaussian Naïve Bayes    0.9722                0.9744             0.9722

   F1 Score (weighted)
0               0.9450
1               0.9723
```

```
Better model based on accuracy: Gaussian Naïve Bayes
```

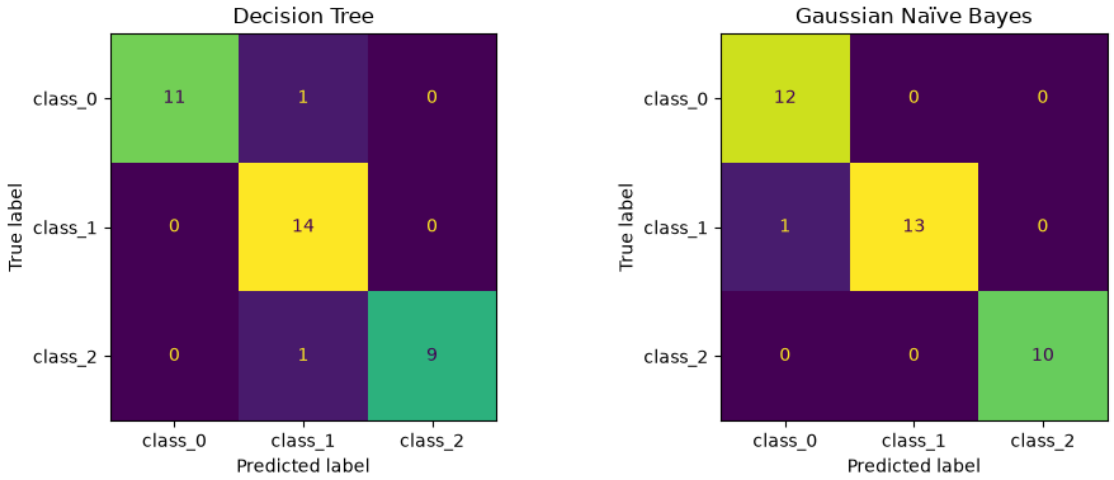
## 6. Confusion Matrices

[7]:
```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

ConfusionMatrixDisplay(
    confusion_matrix(y_test, y_pred_dt),
    display_labels=data.target_names
).plot(ax=axes[0], colorbar=False)
axes[0].set_title("Decision Tree")

ConfusionMatrixDisplay(
    confusion_matrix(y_test, y_pred_nb),
    display_labels=data.target_names
).plot(ax=axes[1], colorbar=False)
axes[1].set_title("Gaussian Naïve Bayes")

plt.tight_layout()
plt.show()
```



## Conclusion

Decision Tree and Gaussian Naïve Bayes classifiers were implemented on the Wine dataset. Both models were evaluated using precision, recall, F1-score, accuracy, and confusion matrices. The comparison table identifies which classifier performed better on the test data.